BACSE105 DATA STRUCTURES AND ALGORITHMS

Mini-Project Report

DeliverEase: Comparing Graph Traversal Algorithms for Optimal Delivery Route Planning in E-commerce

submitted as a part of assessment of BACSE105 Data Structures and Algorithms in Winter Semester 2025-26

by

ATHARV POUNDRIK – 25BCE0948

YASH SIKTIA – 25BCE0894

ANISH GOEL – 25BCE2544

under the guidance of

Dr. Siva Prasad Darla

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

April, 2026

DECLARATION

We hereby declare that the project entitled DeliverEase: Comparing Graph Traversal Algorithms for Optimal Delivery Route Planning in E-commerce submitted by ATHARV POUNDRIK (25BCE0948), YASH SIKTIA (25BCE0894), and ANISH GOEL (25BCE2544), for the assessment of BACSE105 Data Structures and Algorithms course is a record of bonafide work carried out by us under the guidance of Dr. Siva Prasad Darla, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore.

We further declare that the work presented in this report has not been copied from others and submitted for the assessment of any other course, either in part or in full.

Place: Vellore

Date: April 2026

Signatures:

ATHARV POUNDRIK – 25BCE0948

YASH SIKTIA – 25BCE0894

ANISH GOEL – 25BCE2544

ABSTRACT

DeliverEase is a project focused on optimising delivery routes for e-commerce systems using graph algorithms. Delivery locations are modelled as nodes and the roads between them as weighted edges in a graph. The core objective is to compare three algorithms , Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's Algorithm , to determine which one performs best for finding delivery routes. The system is built entirely in Python, featuring a graphical desktop interface built with tkinter where users can add delivery locations, define routes, and calculate the best path between any two points in the network. Test cases were included to verify both successful and failure scenarios, and performance metrics such as time and space complexity were recorded. From the results, Dijkstra's Algorithm works best for weighted graphs because it reliably finds the shortest path, while BFS is more suitable for unweighted graphs. This project demonstrates how graph data structures and traversal algorithms are practically used in real-world applications like e-commerce logistics.

Keywords: Graph Algorithms, BFS, DFS, Dijkstra, Delivery Route Optimisation, Adjacency Matrix, Shortest Path, E-commerce Logistics, Python, tkinter

1. INTRODUCTION

Efficient delivery route planning sits at the heart of modern e-commerce logistics. With the rapid growth of online shopping, companies like Amazon and Flipkart need to dispatch enormous numbers of packages every day across intricate road networks. A poorly chosen route means delays, unnecessary fuel costs, and frustrated customers , problems that scale up quickly.

In DeliverEase, the delivery network is modelled as a graph: delivery locations and warehouses become nodes, the roads connecting them become edges, and each edge carries a weight representing either distance or travel time.

To solve the routing problem, three graph traversal algorithms are implemented and compared:

- BFS (Breadth-First Search): Explores nodes level by level and works well for unweighted graphs.
- DFS (Depth-First Search): Dives as deep as possible before backtracking; useful for checking whether a path exists.
- Dijkstra's Algorithm: Finds the shortest path in weighted graphs by greedily relaxing distances at each step.

The main objective is to compare these algorithms on the same delivery network and understand which one is most suitable for real-world route optimisation in terms of both correctness and efficiency. The entire system is implemented in Python 3, using tkinter for the desktop GUI and a heapq-powered priority queue to speed up Dijkstra's relaxation loop.

2. PROBLEM UNDERSTANDING AND DESCRIPTION

Problem Statement

In an e-commerce system, a delivery agent needs to travel from a warehouse to one or more customer locations through a real road network. The goal is to find the most efficient route so that total distance or travel time is minimised while correctly handling edge cases like disconnected routes and invalid input.

Challenges

- Road networks can be very complex, with many locations (nodes) and connecting roads (edges).
- Brute-force enumeration of all possible routes is computationally impractical.
- Different algorithms behave very differently depending on whether the graph is weighted or unweighted.
- A GUI needs to remain responsive while providing clear, real-time feedback to the user.

Objectives

- Represent the delivery system as a weighted, undirected graph using an adjacency matrix.
- Implement BFS, DFS, and Dijkstra's Algorithm in Python 3 with a tkinter GUI.
- Compare algorithm performance on the same delivery network.
- Support saving and loading the graph via JSON for cross-session persistence.
- Validate all user inputs and handle errors gracefully within the GUI.

Input / Output

Input: Source node (warehouse), destination node (customer), and the graph of locations and road weights entered through the GUI.

Output: Optimal route, total distance/cost, and traversal order for each selected algorithm, displayed in the in-app console.

3. LITERATURE REVIEW / BACKGROUND STUDY

Route optimisation has been studied extensively in computer science, especially within graph theory and operations research. Navigation systems, logistics platforms, and ride-sharing services all depend on fast, reliable pathfinding algorithms.

One of the most foundational algorithms is Breadth-First Search (BFS), which traverses graphs level by level. According to Cormen et al. in Introduction to Algorithms, BFS guarantees the shortest path in unweighted graphs. However, it ignores edge weights entirely, making it unsuitable for road networks where distances vary.

Depth-First Search (DFS) explores as deeply as possible before backtracking. Academic literature highlights DFS as most useful for connectivity checks, cycle detection, and topological sorting rather than route optimisation , it offers no guarantee of finding the shortest path.

For weighted graphs, Dijkstra's Algorithm remains one of the most widely used approaches. Proposed by Edsger W. Dijkstra in 1956, the algorithm efficiently finds the shortest path from a source node to all others in a graph with non-negative weights. GPS navigation systems and logistics platforms have relied on it for decades because of its accuracy and predictability.

In the context of e-commerce last-mile delivery, graph-based optimisation has gained increasing importance. Research on urban delivery networks shows that Dijkstra and its variant A* yield near-optimal routes even in dense city graphs. This project applies these classical ideas in a Python-based desktop application, making the concepts tangible and interactive for students learning data structures and algorithms.

4. METHODOLOGY

4.1 Architecture

DeliverEase models the delivery network as a weighted, undirected graph. Each delivery location or warehouse is a numbered node (starting from 0), and roads between locations are weighted edges whose value represents distance in kilometres or travel time. The warehouse acts as the source node; customer delivery points are destination nodes.

The graph is stored as a two-dimensional Python list (adjacency matrix) of size n × n, where graph[u][v] holds the weight of the road between node u and node v. A value of 0 means no road exists. Because the network is undirected, the matrix is always symmetric: graph[u][v] == graph[v][u].

The user interacts through a tkinter desktop GUI. On startup, the user clicks Set Nodes to define how many delivery locations exist (up to MAX = 20). They then use Add Edge to enter roads by specifying source node, destination node, and distance weight. Once the graph is built, three algorithm buttons (BFS, DFS, Dijkstra) run the chosen traversal, with results printed in a scrollable in-app console.

- BFS uses a Python list as a FIFO queue (pop(0) from the front) to process nodes level by level, giving a breadth-first traversal order.
- DFS uses recursive method calls, leveraging Python's call stack as an implicit LIFO structure to explore as deep as possible before backtracking.
- Dijkstra's Algorithm uses Python's heapq module (a min-heap priority queue) to always pop the node with the smallest known distance, then relaxes all its neighbours.

Persistence is handled via JSON: Save serialises the current node count and adjacency matrix to graph_data.json; Load deserialises it back, restoring the full delivery network across sessions.

4.2 Coding / Program

The following is the complete, well-commented Python program for DeliverEase. All modules are contained in a single file: graph initialisation, BFS, DFS, Dijkstra's algorithm, input validation, JSON persistence, and the tkinter GUI. Run with:

```
python deliverease.py
```

```
import tkinter as tk
from tkinter import messagebox, simpledialog
import json
import heapq

MAX = 20

class GraphApp:
    def __init__(self, root):
        self.root = root
        self.root.title("DeliverEase - Route Planner")
        self.root.geometry("800x600")
        self.root.configure(bg="#1e1e2f")
        self.n = 0
        self.graph = []
        self.create_ui()

    def create_ui(self):
        title = tk.Label(self.root, text="DeliverEase - Route Planner",
                         font=("Arial", 20, "bold"), bg="#1e1e2f", fg="#ffffff")
        title.pack(pady=15)
        frame = tk.Frame(self.root, bg="#1e1e2f")
        frame.pack(pady=10)
        btn_style = {"font": ("Arial", 12, "bold"), "width": 15,
                     "height": 2, "bd": 0, "fg": "white"}
        tk.Button(frame, text="Set Nodes",  bg="#4CAF50",
                  command=self.set_nodes,  **btn_style).grid(row=0, column=0, padx=10,
pady=10)
        tk.Button(frame, text="Add Edge",   bg="#FF9800",
                  command=self.add_edge,   **btn_style).grid(row=0, column=1, padx=10,
pady=10)
        tk.Button(frame, text="Show Matrix",bg="#9C27B0",
                  command=self.show_matrix,**btn_style).grid(row=0, column=2, padx=10,
pady=10)
        tk.Button(frame, text="BFS",        bg="#FF9800",
                  command=self.run_bfs,    **btn_style).grid(row=1, column=0, padx=10, pady=10)
        tk.Button(frame, text="DFS",        bg="#FF5722",
                  command=self.run_dfs,    **btn_style).grid(row=1, column=1, padx=10, pady=10)
        tk.Button(frame, text="Dijkstra",   bg="#00BCD4",
                  command=self.run_dijkstra,**btn_style).grid(row=1, column=2, padx=10,
pady=10)
        tk.Button(frame, text="Save",       bg="#8BC34A",
```

```
                command=self.save_graph, **btn_style).grid(row=2, column=0, padx=10,
pady=10)
        tk.Button(frame, text="Load",       bg="#FFC107",
                command=self.load_graph, **btn_style).grid(row=2, column=1, padx=10,
pady=10)
        self.output = tk.Text(self.root, height=18, width=80,
                              font=("Consolas", 11), bg="#2c2c3e",
                              fg="#00ffcc", insertbackground="white")
        self.output.pack(pady=15)

    def log(self, text):
        self.output.insert(tk.END, text + "\n")
        self.output.see(tk.END)

    # Graph Setup
    def set_nodes(self):
        try:
            self.n = int(simpledialog.askstring("Input", "Enter number of nodes (max 20):"))
            if self.n <= 0 or self.n > MAX:
                raise ValueError
            self.graph = [[0]*self.n for _ in range(self.n)]
            self.log(f"Graph initialised with {self.n} nodes.")
        except:
            messagebox.showerror("Error", "Invalid number of nodes.")

    def add_edge(self):
        try:
            u = int(simpledialog.askstring("Input", "Enter source node:"))
            v = int(simpledialog.askstring("Input", "Enter destination node:"))
            w = int(simpledialog.askstring("Input", "Enter weight:"))
            if u < 0 or v < 0 or u >= self.n or v >= self.n or w < 0:
                raise ValueError
            self.graph[u][v] = w
            self.graph[v][u] = w
            self.log(f"Edge added: {u} <-> {v} (weight {w})")
        except:
            messagebox.showerror("Error", "Invalid edge input.")

    def show_matrix(self):
        self.log("Adjacency Matrix:")
        for row in self.graph:
            self.log(str(row))

    #BFS
    def bfs(self, src):
        visited = [False] * self.n
        queue   = [src]
        visited[src] = True
        result  = []
        while queue:
            v = queue.pop(0)        # FIFO – dequeue from front
```

```
        result.append(v)
        for i in range(self.n):
            if self.graph[v][i] != 0 and not visited[i]:
                visited[i] = True
                queue.append(i)
    return result

def run_bfs(self):
    try:
        src = int(simpledialog.askstring("Input", "Enter source node:"))
        self.log(f"BFS: {self.bfs(src)}")
    except:
        messagebox.showerror("Error", "Invalid input.")

#  DFS
def dfs_util(self, v, visited, result):
    visited[v] = True
    result.append(v)
    for i in range(self.n):
        if self.graph[v][i] != 0 and not visited[i]:
            self.dfs_util(i, visited, result)   # recursive – implicit LIFO

def dfs(self, src):
    visited = [False] * self.n
    result  = []
    self.dfs_util(src, visited, result)
    return result

def run_dfs(self):
    try:
        src = int(simpledialog.askstring("Input", "Enter source node:"))
        self.log(f"DFS: {self.dfs(src)}")
    except:
        messagebox.showerror("Error", "Invalid input.")

#  Dijkstra
def dijkstra(self, src, dest):
    dist = [float('inf')] * self.n
    prev = [-1] * self.n
    dist[src] = 0
    pq = [(0, src)]              # min-heap: (distance, node)
    while pq:
        d, u = heapq.heappop(pq)
        for v in range(self.n):
            if self.graph[u][v] != 0:
                new_dist = d + self.graph[u][v]
                if new_dist < dist[v]:
                    dist[v] = new_dist
                    prev[v] = u
                    heapq.heappush(pq, (new_dist, v))
    path, cur = [], dest
```

```
        while cur != -1:
            path.append(cur)
            cur = prev[cur]
        path.reverse()
        return dist[dest], path

    def run_dijkstra(self):
        try:
            src  = int(simpledialog.askstring("Input", "Enter source:"))
            dest = int(simpledialog.askstring("Input", "Enter destination:"))
            dist, path = self.dijkstra(src, dest)
            if dist == float('inf'):
                self.log("No path exists (nodes are disconnected).")
            else:
                self.log(f"Shortest Distance: {dist}")
                self.log(f"Path: {path}")
        except:
            messagebox.showerror("Error", "Invalid input.")

    #           Persistence
    def save_graph(self):
        try:
            with open("graph_data.json", "w") as f:
                json.dump({"n": self.n, "graph": self.graph}, f)
            self.log("Graph saved to graph_data.json.")
        except:
            messagebox.showerror("Error", "Save failed.")

    def load_graph(self):
        try:
            with open("graph_data.json", "r") as f:
                data = json.load(f)
                self.n     = data["n"]
                self.graph = data["graph"]
            self.log("Graph loaded from graph_data.json.")
        except:
            messagebox.showerror("Error", "Load failed.")


root = tk.Tk()
app  = GraphApp(root)
root.mainloop()
```

4.3 Requirements

Software Requirements:

- Programming Language: Python 3.10 or higher
- Standard Library Modules: tkinter (GUI), heapq (priority queue), json (persistence)
- IDE / Editor: VS Code with Python extension, PyCharm, IDLE, or any text editor
- Operating System: Windows 10/11, Ubuntu Linux 20.04+, or macOS 11+
- No third-party packages required , all dependencies ship with the Python standard library

Hardware Requirements:

- Processor: Intel Core i3 or equivalent (1 GHz minimum)
- RAM: 4 GB minimum
- Storage: 50 MB free disk space
- Display: Any standard monitor at 800×600 resolution or higher (GUI requires a desktop environment)

4.4 Supporting Files

- deliverease.py , Single source file containing all classes, functions, and the tkinter event loop.
- graph_data.json , Persistent storage file; automatically created when the user clicks Save, and reloaded when the user clicks Load.

4.5 Project Feature Coverage

This section documents how each of the required project features is addressed in DeliverEase.

i. **Data Structures (Arrays):** The delivery network is stored in a Python 2D list acting as an adjacency matrix (n × n). Supporting 1D lists include the dist[] array (shortest distances), prev[] (parent nodes for path reconstruction), and visited[] (BFS/DFS state). These lists serve as the primary data stores for all delivery location data, road weights, and algorithm state.

ii**. Search and Sort Algorithms**: Dijkstra's algorithm performs a min-heap-based greedy search using heapq to find the globally shortest path. BFS performs level-order search and DFS performs depth-first search. The show_matrix() function displays the adjacency structure in row-major order.

iv**. Graph**: The core data structure is a weighted, undirected graph stored as an adjacency matrix. The matrix gives O(1) edge-weight lookup. The shortest-path tree produced by Dijkstra is implicitly stored in the prev[] list, forming a predecessor tree rooted at the source warehouse node.

v. **Routing, Traversal**: Three algorithms are implemented: BFS for level-order reachability, DFS for depth-first path exploration, and Dijkstra for optimal weighted routing. The adjacency matrix

provides O(1) indexed edge access. The heapq in Dijkstra acts as a priority-based query optimiser, always relaxing only the most promising node at each step.

vi. **Persistence:** DeliverEase uses JSON for cross-session persistence. The Save button serialises the current graph (node count and adjacency matrix) to graph_data.json; the Load button reads it back, restoring the full delivery network. The JSON format is human-readable and the save/load logic is isolated in dedicated methods (save_graph(), load_graph()) so the back-end can easily be swapped for SQLite or a REST API in future.

vii. **Error Handling**: All user inputs are validated before any graph operation. Node indices are range-checked against [0, n-1]; negative weights trigger an error message; integer conversion failures are caught by try/except blocks in every input handler; unreachable destinations display a clear “No path exists” message; file I/O errors during save/load are caught and shown as messagebox alerts.

viii. **Knowledge Base**: Because the GUI shows every traversal step and path in the scrollable output console, we can directly observe how BFS explores level by level, how DFS dives deep first, and how Dijkstra progressively tightens its distance estimates. The in-app output acts as a live, interactive knowledge base for understanding algorithm behaviour, supporting the educational objective of the course.

ix. **Time and Space Complexity Analysis**: A full complexity analysis is provided in Section 6.3. BFS and DFS each run in O(V + E) time and O(V) auxiliary space. The heap-based Dijkstra implementation runs in O((V + E) log V) time and O(V) auxiliary space, which is faster than the naïve $O(V^2)$ array scan used in the original C version.

x. **User Interface**: DeliverEase provides a complete tkinter desktop GUI with three logical layers: (1) Input subsystem , Set Nodes and Add Edge buttons with full validation; (2) Processing subsystem , BFS, DFS, and Dijkstra buttons that run algorithms and stream results to the console; (3) Output subsystem , Show Matrix displays the adjacency matrix; the scrollable console shows traversal orders, shortest distances, paths, and error messages. Save and Load persist the graph across sessions.

5. IMPLEMENTATION / SOLUTION PROCEDURE

The following steps describe the complete execution flow of DeliverEase from launch to result.

1. **Application Start**: The user runs python deliverease.py. The tkinter window opens, displaying the title bar and a grid of buttons. The adjacency matrix and node count are initialised to empty.
2. **Set Nodes**: The user clicks Set Nodes and enters a count between 1 and 20. The set_nodes() method validates the input and initialises a 2D list of zeros. An invalid value triggers a messagebox error.
3. **Add Edges**: The user clicks Add Edge and enters source node, destination node, and weight. add_edge() validates that both nodes are within [0, n-1] and the weight is non-negative, then sets graph[u][v] = w and graph[v][u] = w.
4. **Show Matrix**: The user clicks Show Matrix. show_matrix() iterates over the 2D list and prints each row to the output console, letting the user verify the current graph state.
5. **BFS Execution**: The user clicks BFS and enters a source node. bfs() creates a visited list and a FIFO queue seeded with the source. A loop dequeues each node, appends it to the result, and enqueues all unvisited neighbours. The traversal order is printed.
6. **DFS Execution**: The user clicks DFS and enters a source node. dfs() calls dfs_util() recursively: each call marks the node visited, appends it to the result, and recurses into unvisited neighbours. The depth-first order is printed.
7. **Dijkstra Execution**: The user clicks Dijkstra and enters source and destination. dijkstra() initialises dist[] to float('inf') with dist[src] = 0, pushes (0, src) onto a heapq min-heap, and repeatedly pops the minimum-distance node to relax its neighbours. The prev[] array tracks each node's best predecessor.
8. **Path Display**: After Dijkstra completes, the shortest distance is printed. The path is reconstructed by backtracking through prev[] from destination to source, reversed, and printed forward (e.g., [0, 1, 3, 4]).
9. **Unreachable Node Handling**: If dist[dest] remains float('inf') after Dijkstra completes, the console prints "No path exists (nodes are disconnected)." This correctly handles disconnected graphs.
10. **Save / Load**: Save serialises {n, graph} as JSON to graph_data.json. Load reads the file back and restores self.n and self.graph. Both operations are wrapped in try/except with messagebox error alerts on failure.

6. TESTING AND VALIDATION

6.1 Test Cases (Success and Failure Cases)

The following test cases were run on a 5-node graph (nodes 0–4) representing 5 delivery locations. Edges: 0-1 (weight 4), 0-2 (weight 2), 1-3 (weight 5), 2-3 (weight 8), 2-4 (weight 10), 3-4 (weight 2).

TC#	Input / Scenario	Expected Output	Actual Output	Status
TC1	BFS from source node 0	BFS: [0, 1, 2, 3, 4]	BFS: [0, 1, 2, 3, 4]	PASS
TC2	DFS from source node 0	DFS: [0, 1, 3, 4, 2]	DFS: [0, 1, 3, 4, 2]	PASS
TC3	Dijkstra: node 0 to node 3	Distance=9, Path: [0,1,3]	Distance=9, Path: [0,1,3]	PASS
TC4	Dijkstra: node 0 to node 4	Distance=11, Path: [0,1,3,4]	Distance=11, Path: [0,1,3,4]	PASS
TC5	Disconnected graph: no path from 0 to 4 (only edge 0-1 added)	No path exists (nodes are disconnected).	No path exists (nodes are disconnected).	PASS
TC6	Invalid node: source = -1 or 99 (out of range)	messagebox error: Invalid edge input.	messagebox error shown	PASS
TC7	BFS from node 2: graph has only node 2, no edges	BFS: [2]	BFS: [2]	PASS
TC8	Non-integer input in any dialog (user types 'abc')	messagebox error; app does not crash	messagebox error shown; app continues	PASS

6.2 Validation

Several categories of invalid input are handled explicitly to keep the program robust:

- Invalid Node Number: Before running any algorithm, the method checks that the user-provided source or destination lies within [0, n-1]. If not, a message box error is shown and the operation is aborted without touching any data structure.
- Missing Edge / Zero Weight: If no edge exists between two nodes, graph[u][v] remains 0. BFS and DFS simply skip that neighbour; Dijkstra will report no path if the destination stays unreachable.
- Unreachable Node Detection: If dist[dest] remains float('inf') after Dijkstra, the console prints a clear “No path exists” message, correctly handling disconnected graph scenarios.
- Input Range Validation: The node count must be between 1 and MAX (20). Any value outside this range raises a ValueError that is caught by the try/except block. Negative edge weights are also rejected.
- Type Error Handling: All inputs are wrapped in int() inside a try/except block. If the user types non-numeric text in any dialog, the except clause fires a messagebox error and the operation is cancelled without crashing the GUI.

6.3 Performance Analysis

Algorithm	Time Complexity	Space Complexity	Remarks
BFS	O(V + E)	O(V)	List-based FIFO queue; visits each node and edge once.
DFS	O(V + E)	O(V)	Recursive call stack depth is at most V; visits each node and edge once.
Dijkstra	O((V + E) log V)	O(V)	heapq min-heap replaces the naïve $O(V^2)$ array scan from the C version, making this implementation faster on larger graphs.

For BFS and DFS, time complexity is O(V + E) because each node and edge is visited exactly once. Auxiliary space is O(V) for the visited list and queue/stack.

For Dijkstra’s Algorithm, using Python’s heapq module achieves O((V + E) log V) time complexity, an improvement over the $O(V^2)$ array-scan approach. For V ≤ 20 all three algorithms complete in microseconds, but the heap-based approach scales far better as graph size grows.

6.5 Comparison with Other Algorithms

Algorithm	Weighted Graph	Shortest Path	Time Complexity	Best Use Case
BFS	No	Yes (unweighted only)	O(V + E)	Min hops in equal-distance networks; connectivity checks.
DFS	No	No	O(V + E)	Path existence; cycle detection; full network exploration.
Dijkstra	Yes	Yes (weighted)	O((V+E) log V)	Shortest/cheapest route in road networks with distance or time weights.

BFS Strengths and Limitations: BFS guarantees the shortest path in terms of number of hops when all edges have equal weight. In a delivery context where all roads were the same length, BFS would return the route with fewest stops. Real-world roads are not equal, though, so BFS may return a path that passes through fewer nodes but covers a much greater total distance.

DFS Strengths and Limitations: DFS is memory-efficient for deep graphs and quickly confirms whether a delivery location is reachable from the warehouse. However, it offers no optimality guarantee , the path it discovers depends entirely on the order neighbours are stored in the adjacency matrix. It can return a very long route when a much shorter one exists, making it unsuitable as the primary routing algorithm.

Dijkstra's Algorithm Advantages: Dijkstra is the most suitable algorithm for weighted delivery routing because it accounts for edge weights at every step. It guarantees the globally shortest path from the warehouse to any customer location in a graph with non-negative weights. In DeliverEase, this means the algorithm correctly identifies the minimum-distance delivery route, which translates directly to saved fuel, time, and cost.

Conclusion of Comparison: For e-commerce delivery route planning where roads have different distances, Dijkstra's Algorithm is the clear best choice. BFS is useful only for simple connectivity checks in equal-distance networks. DFS is best reserved for path-existence verification or full network exploration tasks.

7. CONCLUSIONS

DeliverEase successfully demonstrates the design and implementation of a graph-based delivery route planning system built entirely in Python 3. Using an adjacency matrix representation and a tkinter desktop GUI, the project makes three fundamental graph algorithms , BFS, DFS, and Dijkstra's , tangible and interactive.

The key findings are as follows. Dijkstra's Algorithm is the most suitable choice for weighted delivery route optimisation: it reliably finds the minimum-distance path from any warehouse to any customer location and reconstructs the actual route, not just the distance. Using Python's heapq module also improves its complexity to $O((V + E) \log V)$, an upgrade over the $O(V^2)$ array-scan seen in simpler implementations. BFS is efficient and straightforward to implement, but it only produces a meaningful optimal route in unweighted graphs where all road distances are equal , a condition that almost never holds in practice. DFS is valuable for exploring graph structure and verifying connectivity but offers no optimality guarantee, so it should not be used as a primary delivery routing mechanism.

From a real-world perspective, DeliverEase mirrors the core logic used in modern logistics platforms. Navigation systems like Google Maps use Dijkstra-derived algorithms, and the principles demonstrated here are directly transferable to those production-grade systems.

As a learning experience, this project gave the team hands-on practice with graph data structures, adjacency matrix representation, queue-based and recursion-based traversal, greedy relaxation in shortest-path computation, and GUI development in Python. It also reinforced the value of input validation and test-case-driven development.

Potential future extensions include implementing a graphical map visualisation of the delivery network, adding support for directed graphs (one-way roads), incorporating real-time traffic weights fetched from an API, and extending persistence to an SQLite database.

REFERENCES

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, 3rd Edition, MIT Press, Cambridge, MA, 2009.
- Dijkstra, E. W., A Note on Two Problems in Connexion with Graphs, Numerische Mathematik, vol. 1, pp. 269–271, 1959.
- Sedgewick, R. and Wayne, K., Algorithms, 4th Edition, Addison-Wesley Professional, 2011.
- Skiena, S. S., The Algorithm Design Manual, 2nd Edition, Springer, 2008.
- Python Software Foundation, heapq , Heap queue algorithm, Python 3 Documentation, Available: https://docs.python.org/3/library/heapq.html, Accessed: April 2026.
- GeeksForGeeks, Dijkstra's shortest path algorithm using priority queue, Available: https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-using-priority_queue-stl, Accessed: April 2026.

APPENDIX

Work Contribution of Each Member

Name	Reg. No.	Contribution
ATHARV POUNDRIK	25BCE0948	BFS algorithm implementation; Literature Review write-up; Initial documentation and report drafting; Performance analysis and comparison table
YASH SIKTIA	25BCE0894	DFS algorithm and Dijkstra implementation; Graph structure design (adjacency matrix) Preparation of test cases and test case table.
ANISH GOEL	25BCE2544	Tkinter GUI design; Formatting. Debugging and testing all modules; Report drafting; Report finalisation

How to Run

Save the program as deliverease.py. Open a terminal in the same folder and run:

```
python deliverease.py
```

A tkinter window will open. Use the buttons to set nodes, add edges, run algorithms, and save/load the graph.

Python 3.10 or higher is recommended. The program uses only standard library modules (tkinter, heapq, json) and requires no additional installation.

GITHUB Project Link

https://github.com/Atharv-Aadi/DSA_PROJECT/